

Deploying Machine Learning Models on AWS SageMaker

A Step-by-Step Deployment Guide

Prabal

April 18, 2026

Contents

1	Introduction	2
2	Core Concepts	2
3	Prerequisites	2
4	Step-by-Step Deployment	3
4.1	Step 1 — Package the Model Artifact	3
4.2	Step 2 — Upload the Artifact to S3	3
4.3	Step 3 — Write the Inference Script	3
4.4	Step 4 — Choose a Container Image	4
4.5	Step 5 — Create the SageMaker Model	5
4.6	Step 6 — Deploy the Endpoint	5
4.7	Step 7 — Invoke the Endpoint	5
4.8	Step 8 — Configure Autoscaling	6
4.9	Step 9 — Monitor and Log	6
4.10	Step 10 — Update, Roll Back, or Delete	7
5	Deployment Variants	7
5.1	Serverless Inference	7
5.2	Asynchronous Inference	7
5.3	Batch Transform	8
5.4	Multi-Model Endpoints (MME)	8
5.5	Multi-Container Endpoints	8
6	Production Best Practices	8
7	Troubleshooting Checklist	8
8	Summary	9

1 Introduction

Amazon SageMaker is AWS's fully managed machine learning service that covers the entire ML lifecycle: data labeling, training, tuning, deployment, and monitoring. For deployment specifically, SageMaker abstracts away the heavy lifting of provisioning compute, setting up HTTPS endpoints, load balancing, autoscaling, and rolling updates, letting teams ship production models with a few API calls.

This document focuses on the **deployment** phase, assuming a model has already been trained (in SageMaker, locally, or elsewhere). It walks through the end-to-end flow of exposing that model as a production inference service.

Note

SageMaker offers four deployment modes:

- **Real-time inference** — persistent HTTPS endpoint, low latency.
- **Serverless inference** — pay-per-request, scales to zero.
- **Asynchronous inference** — queued requests, large payloads, long timeouts.
- **Batch transform** — offline bulk scoring over S3 datasets.

The main flow below targets *real-time inference*, which is the most common mode.

2 Core Concepts

Before deploying, understand the three SageMaker primitives that compose an endpoint:

Primitive	Purpose
Model	A logical object that binds a <i>model artifact</i> (weights stored in S3 as <code>model.tar.gz</code>) to an <i>inference container image</i> (from ECR) and an <i>IAM role</i> .
EndpointConfig	Declares one or more <i>production variants</i> — which model(s) to serve, instance type, instance count, autoscaling policy, and traffic split.
Endpoint	The live, invocable HTTPS resource. Built from an EndpointConfig . Updating it performs a safe blue/green rollout.

Tip

Keep these three objects separate in your mind: **what to serve** (Model), **how to serve it** (EndpointConfig), and **where it is served** (Endpoint). Blue/green deployments and A/B traffic splits all happen by creating new EndpointConfigs and updating the Endpoint to point at them.

3 Prerequisites

1. An AWS account with billing enabled.
2. AWS CLI installed and configured (`aws configure`).
3. Python 3.9+ with `boto3` and `sagemaker` SDK installed:

```
pip install boto3 sagemaker
```

4. An IAM execution role for SageMaker with the `AmazonSageMakerFullAccess` managed policy, plus S3 read access to your model bucket. The role's trust policy must allow `sagemaker.amazonaws.com` to assume it.
5. An S3 bucket in the same region as your intended endpoint.
6. A trained model (e.g., a PyTorch `.pt` file, TensorFlow `SavedModel`, scikit-learn pickle, or HuggingFace checkpoint).

Important

Region consistency matters. The IAM role, S3 bucket, ECR image, and SageMaker endpoint must all be in the *same AWS region*. Cross-region references cause opaque permission errors and latency overhead.

4 Step-by-Step Deployment

4.1 Step 1 — Package the Model Artifact

SageMaker expects the model artifact as a gzipped tarball named `model.tar.gz`. The internal layout depends on the framework, but the general structure for a PyTorch model is:

```
model/  
  code/  
    inference.py      # entry point (see Step 3)  
    requirements.txt  # extra dependencies (optional)  
    model.pth         # the trained weights
```

Create the tarball:

```
cd model/  
tar -czvf model.tar.gz *
```

Note

For built-in algorithms (XGBoost, Linear Learner, etc.) SageMaker's own training jobs produce a compatible `model.tar.gz` automatically — you can skip the custom packaging step.

4.2 Step 2 — Upload the Artifact to S3

```
aws s3 cp model.tar.gz s3://my-ml-artifacts/models/sentiment-v1/model.tar.gz
```

Note the S3 URI — it will be referenced when creating the SageMaker Model object. Use versioned paths (e.g., `sentiment-v1`, `sentiment-v2`) so rollbacks are trivial.

4.3 Step 3 — Write the Inference Script

For custom models, SageMaker's framework containers (PyTorch, TensorFlow, HuggingFace, scikit-learn) expose four overridable hooks in `inference.py`:

```
1 import json  
2 import os  
3 import torch  
4  
5 def model_fn(model_dir):  
6     """Load and return the model. Called once when the container starts."""
```

```

7     model = torch.load(os.path.join(model_dir, "model.pth"),
8                           map_location="cpu")
9     model.eval()
10    return model
11
12    def input_fn(request_body, request_content_type):
13        """Parse the incoming request payload."""
14        if request_content_type == "application/json":
15            data = json.loads(request_body)
16            return torch.tensor(data["inputs"], dtype=torch.float32)
17        raise ValueError(f"Unsupported content type: {request_content_type}")
18
19    def predict_fn(input_data, model):
20        """Run inference."""
21        with torch.no_grad():
22            return model(input_data)
23
24    def output_fn(prediction, response_content_type):
25        """Serialize the prediction."""
26        if response_content_type == "application/json":
27            return json.dumps({"predictions": prediction.tolist()})
28        raise ValueError(f"Unsupported content type: {response_content_type}")

```

Tip

If any of these four functions are missing, SageMaker uses sensible defaults that handle JSON / NumPy payloads. For quick prototypes you can often ship with just `model_fn`.

4.4 Step 4 — Choose a Container Image

There are three options:

1. **Built-in SageMaker images** — Maintained by AWS for PyTorch, TensorFlow, MXNet, scikit-learn, XGBoost, HuggingFace, etc. Resolve the URI via the SDK:

```

1 from sagemaker import image_uris
2 image_uri = image_uris.retrieve(
3     framework="pytorch",
4     region="us-east-1",
5     version="2.1.0",
6     py_version="py310",
7     image_scope="inference",
8     instance_type="ml.m5.xlarge",
9 )

```

2. **Extend a SageMaker image** — Start FROM an official image and add custom system packages or Python libraries. Push to ECR.
3. **Bring Your Own Container (BYOC)** — Fully custom Docker image that implements the SageMaker inference contract (exposes `/ping` and `/invocations` on port 8080). Push to ECR.

Important

If you use BYOC, the container must respond to GET `/ping` with HTTP 200 within the timeout (default 60 s) or the endpoint will fail to come up. This is the most common BYOC bug.

4.5 Step 5 — Create the SageMaker Model

Using the high-level SageMaker Python SDK:

```
1 from sagemaker.pytorch import PyTorchModel
2 import sagemaker
3
4 sess = sagemaker.Session()
5 role = "arn:aws:iam::123456789012:role/SageMakerExecutionRole"
6
7 model = PyTorchModel(
8     model_data="s3://my-ml-artifacts/models/sentiment-v1/model.tar.gz",
9     role=role,
10    framework_version="2.1.0",
11    py_version="py310",
12    entry_point="inference.py",
13    source_dir="model/code",           # where inference.py lives locally
14    name="sentiment-model-v1",
15    sagemaker_session=sess,
16 )
```

Behind the scenes this resolves the container image, uploads `source_dir` as a secondary artifact, and prepares a Model object — but does not yet create it in the account until `deploy()` is called.

4.6 Step 6 — Deploy the Endpoint

A single call creates the EndpointConfig and Endpoint:

```
1 predictor = model.deploy(
2     initial_instance_count=1,
3     instance_type="ml.m5.xlarge",
4     endpoint_name="sentiment-endpoint",
5 )
```

Provisioning typically takes **5–10 minutes**. The endpoint state transitions: **Creating** → **InService**.

Common instance families:

- `ml.t2.*`, `ml.m5.*` — CPU, general purpose, cheap.
- `ml.c5.*` — compute-optimized CPU, good for latency-sensitive classical ML.
- `ml.g4dn.*`, `ml.g5.*` — NVIDIA T4 / A10G GPUs, cost-effective deep learning inference.
- `ml.p4d.*`, `ml.p5.*` — A100 / H100 GPUs, LLMs and large vision models.
- `ml.inf1.*`, `ml.inf2.*` — AWS Inferentia custom silicon, lowest cost-per-inference for supported models.

4.7 Step 7 — Invoke the Endpoint

From the SDK:

```
1 result = predictor.predict({"inputs": [[0.1, 0.4, 0.9]]})
2 print(result)
```

Or from any language using `boto3` / AWS SDK:

```
1 import boto3, json
2
3 runtime = boto3.client("sagemaker-runtime", region_name="us-east-1")
4 resp = runtime.invoke_endpoint(
5     EndpointName="sentiment-endpoint",
6     ContentType="application/json",
```

```

7     Body=json.dumps({"inputs": [[0.1, 0.4, 0.9]]}),
8 )
9 print(json.loads(resp["Body"].read()))

```

Note

SageMaker endpoints are *private by default* — they sit behind IAM, not the public internet. To expose them externally, put an API Gateway + Lambda (or direct API Gateway HTTP integration) in front, terminating auth there.

4.8 Step 8 — Configure Autoscaling

Autoscaling is *not* enabled by default. Attach a target-tracking policy via Application Auto Scaling:

```

1 import boto3
2
3 asg = boto3.client("application-autoscaling")
4 resource_id = "endpoint/sentiment-endpoint/variant/AllTraffic"
5
6 asg.register_scalable_target(
7     ServiceNamespace="sagemaker",
8     ResourceId=resource_id,
9     ScalableDimension="sagemaker:variant:DesiredInstanceCount",
10    MinCapacity=1,
11    MaxCapacity=8,
12 )
13
14 asg.put_scaling_policy(
15     PolicyName="sentiment-scale",
16     ServiceNamespace="sagemaker",
17     ResourceId=resource_id,
18     ScalableDimension="sagemaker:variant:DesiredInstanceCount",
19     PolicyType="TargetTrackingScaling",
20     TargetTrackingScalingPolicyConfiguration={
21         "TargetValue": 70.0,
22         "PredefinedMetricSpecification": {
23             "PredefinedMetricType": "SageMakerVariantInvocationsPerInstance"
24         },
25         "ScaleInCooldown": 300,
26         "ScaleOutCooldown": 60,
27     },
28 )

```

4.9 Step 9 — Monitor and Log

SageMaker emits metrics to CloudWatch automatically:

- Invocations, InvocationsPerInstance
- ModelLatency, OverheadLatency
- Invocation4XXErrors, Invocation5XXErrors
- CPUUtilization, GPUUtilization, MemoryUtilization

Container stdout/stderr flows to CloudWatch Logs under /aws/sagemaker/Endpoints/<endpoint-name>

For production health, also enable:

- **Model Monitor** — detects data drift, feature skew, model quality drift, and bias over time.
- **Clarify** — explainability and bias reports.
- **CloudWatch Alarms** — page on ModelLatency p99 or 5XX rate.

4.10 Step 10 — Update, Roll Back, or Delete

Rolling update. To ship a new model version, create a new Model and a new EndpointConfig, then update the existing Endpoint:

```
1 sm = boto3.client("sagemaker")
2 sm.update_endpoint(
3     EndpointName="sentiment-endpoint",
4     EndpointConfigName="sentiment-config-v2",
5 )
```

SageMaker performs a **blue/green** swap: the new fleet is provisioned, health-checked, then traffic is shifted. The old fleet is torn down only after the new fleet is healthy.

Delete. Clean up to stop incurring charges (real-time endpoints bill per second regardless of traffic):

```
1 predictor.delete_endpoint()
2 predictor.delete_model()
```

Important

A running `ml.g5.xlarge` endpoint costs roughly \$1+/hour even when idle. **Always delete test endpoints** or use Serverless Inference for development workloads.

5 Deployment Variants

5.1 Serverless Inference

No instance management; pay per invocation and duration. Ideal for spiky or low-traffic workloads.

```
1 from sagemaker.serverless import ServerlessInferenceConfig
2
3 serverless_config = ServerlessInferenceConfig(
4     memory_size_in_mb=4096,
5     max_concurrency=20,
6 )
7 predictor = model.deploy(serverless_inference_config=serverless_config)
```

Trade-offs: cold starts (hundreds of ms to seconds), 6 GB memory ceiling, no GPU support.

5.2 Asynchronous Inference

Requests are queued in S3; responses returned via SNS notifications. Suitable for large payloads (up to 1 GB) or long-running inferences (up to 1 hour).

```
1 from sagemaker.async_inference import AsyncInferenceConfig
2
3 async_config = AsyncInferenceConfig(
4     output_path="s3://my-ml-artifacts/async-outputs/",
5     max_concurrent_invocations_per_instance=4,
6 )
7 predictor = model.deploy(
8     initial_instance_count=1,
9     instance_type="ml.m5.xlarge",
10    async_inference_config=async_config,
11 )
```

5.3 Batch Transform

One-shot bulk scoring of a dataset in S3, no persistent endpoint:

```
1 transformer = model.transformer(  
2     instance_count=2,  
3     instance_type="ml.m5.xlarge",  
4     output_path="s3://my-ml-artifacts/batch-outputs/",  
5 )  
6 transformer.transform(  
7     data="s3://my-ml-artifacts/batch-inputs/",  
8     content_type="application/json",  
9 )  
10 transformer.wait()
```

5.4 Multi-Model Endpoints (MME)

Host *many* models behind one endpoint, loaded dynamically from S3 on demand. Ideal when you have hundreds or thousands of small models (e.g., one per customer) with sparse traffic.

5.5 Multi-Container Endpoints

Host multiple containers behind one endpoint. Invoke a specific container by name, or chain them in a serial inference pipeline (e.g., preprocessing → model → postprocessing).

6 Production Best Practices

1. **Infrastructure as Code** — Define endpoints in CloudFormation, CDK, or Terraform. Avoid console-driven deployments.
2. **CI/CD** — Use SageMaker Pipelines or CodePipeline for train → evaluate → register → deploy flows. Store approved models in the *Model Registry*.
3. **Blue/Green + Canary** — Use `DeploymentConfig` with canary or linear traffic shifting for risk-averse rollouts with automatic CloudWatch-alarm-triggered rollback.
4. **VPC isolation** — Place endpoints in a VPC with no internet egress when handling sensitive data. Use VPC endpoints for S3 and ECR pulls.
5. **Encryption** — Enable KMS encryption for model artifacts in S3, endpoint EBS volumes, and inter-node traffic.
6. **Right-sizing** — Use SageMaker Inference Recommender to benchmark instance types against your latency and throughput SLOs.
7. **Shadow testing** — Deploy new variants with 0% traffic and mirror production requests to validate before cutover.
8. **Cost hygiene** — Tag endpoints with project / environment / owner and review Cost Explorer weekly. Use Savings Plans for steady workloads.

7 Troubleshooting Checklist

Symptom	Likely Cause / Fix
---------	--------------------

Endpoint stuck in <code>Creating</code> then <code>Failed</code>	Check CloudWatch logs. Usually <code>model_fn</code> throws, container <code>/ping</code> fails, or <code>requirements.txt</code> install times out.
<code>AccessDenied</code> on S3	SageMaker execution role missing <code>s3:GetObject</code> on the artifact bucket.
<code>ModelError: 413 Payload Too Large</code>	Real-time endpoints cap payloads at 6 MB. Use Async Inference for larger inputs.
<code>ModelLatency</code> spikes	Enable autoscaling; increase instance size; batch requests; consider GPU or Inferentia.
Cold start on first invocation	Pre-warm by invoking after deploy; use provisioned concurrency on serverless.
High cost with low traffic	Switch to Serverless Inference or Async Inference; delete idle endpoints.

8 Summary

The canonical SageMaker real-time deployment flow is:

1. Package model $\rightarrow$ `model.tar.gz`.
2. Upload to S3.
3. Write `inference.py` (four hooks).
4. Pick or build a container image.
5. Create a `Model` (binds artifact + image + role).
6. Create an `EndpointConfig` (instance type, count, scaling).
7. Create / update an `Endpoint`.
8. Invoke via `sagemaker-runtime`.
9. Attach autoscaling and CloudWatch alarms.
10. Monitor with Model Monitor; iterate with blue/green updates.

The SDK collapses steps 5–7 into a single `model.deploy()` call, which is why SageMaker feels lightweight in practice despite the rich underlying primitives.